순차논리회로(Latch 및 Flip-Flop) 설계 실습 결과 보고서

과목명: 디지털논리회로 및 실습 (Logisim / QuestaSim) **학번 / 이름:** 2023001421 / 허은찬

1. 실습 목적 및 진행 방식

이번 실습에서는 순차논리회로의 가장 기본이 되는 기억 소자인 Latch와 Flip-Flop 5종류(RS Latch, D Latch, D Flip-Flop, JK Flip-Flop, 비동기 입력이 추가된 D Flip-Flop)를 직접 설계하고 동작을 확인하였다. Logisim에서 제공하는 기본 메모리 소자를 쓰지 않고 NOR, NAND, NOT 게이트만으로 피드백 루프를 직접 배선해서 회로를 구성했으며, 같은 동작을 하는 VHDL 코드를 함께 작성해 두 회로의 출력 결과가 똑같이 나오는지를 비교하며 테스트를 진행했다.

2. NOR 게이트 기반 RS Latch

2.1 회로 구성 및 동작 원리

RS Latch는 2입력 NOR 게이트 2개를 서로 교차 피드백(Cross-coupled) 형태로 연결해서 만들었다. NOR 게이트는 입력 중 하나라도 1이 들어가면 출력이 무조건 0이 되기 때문에, s=1, r=0을 넣으면 위쪽 NOR 게이트 출력인 nq가 먼저 0이 되고, 이 값이 아래쪽 게이트로 넘어가서 q=1로 세트된다. 반대로 s=1, r=1을 동시에 넣으면 두 출력이 모두 0으로 떨어져서 q와 nq가 서로 반대 값을 가져야 한다는 조건이 깨지므로 금지 상태(Forbidden)가 된다.

입력 S	입력 R	출력 Q	출력 NQ	동작 상태
0	0	이전 값 유지 (초기 E)	이전 값 유지 (초기 E)	Hold (유지) - 처음 시뮬레이션을 켰을 때는 초기값이 없어서 E(VHDL은 U)가 뜸
1	0	1	0	Set (설정) - 출력 q가 1로 설정됨
0	1	0	1	Reset (초기화) - 출력 q가 0으로 리셋됨
1	1	0	0	Forbidden (금지 상태) - 두 출력이 모두 0이 됨

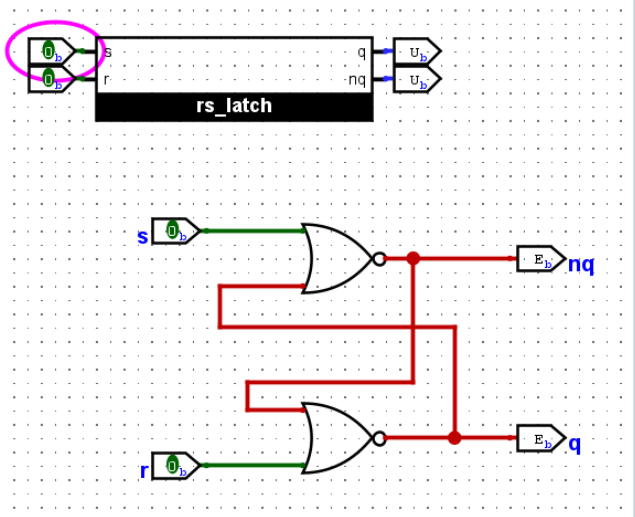
2.2 VHDL 코드 (rs_latch.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

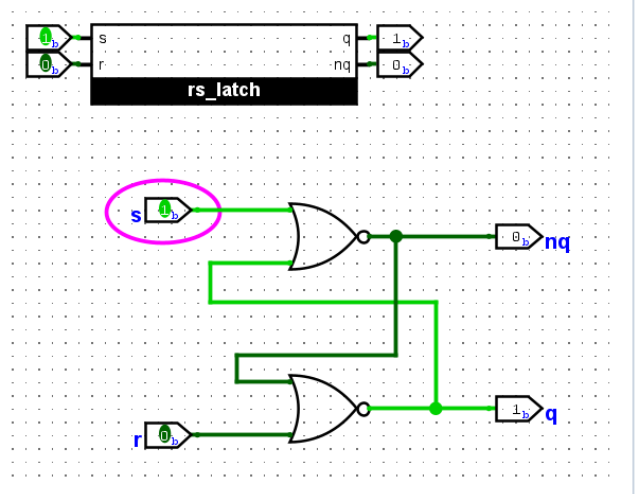
ENTITY rs_latch IS
    PORT (
        s  : IN  std_logic;
        r  : IN  std_logic;
        q  : OUT std_logic;
        nq : OUT std_logic
    );
END rs_latch;

ARCHITECTURE bhv OF rs_latch IS
    SIGNAL q_int  : std_logic;
    SIGNAL nq_int : std_logic;
BEGIN
    PROCESS (s, r, q_int, nq_int)
    BEGIN
        IF (s = '1' AND r = '1') THEN
            q_int  <= '0';
            nq_int <= '0';
        ELSIF (s = '1' AND r = '0') THEN
            q_int  <= '1';
            nq_int <= '0';
        ELSIF (s = '0' AND r = '1') THEN
            q_int  <= '0';
            nq_int <= '1';
        END IF;
    END PROCESS;
    q  <= q_int;
    nq <= nq_int;
END bhv;
```

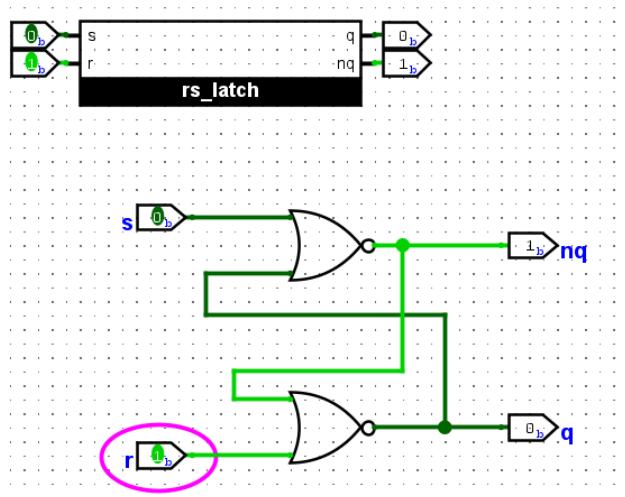
2.3 시뮬레이션 결과



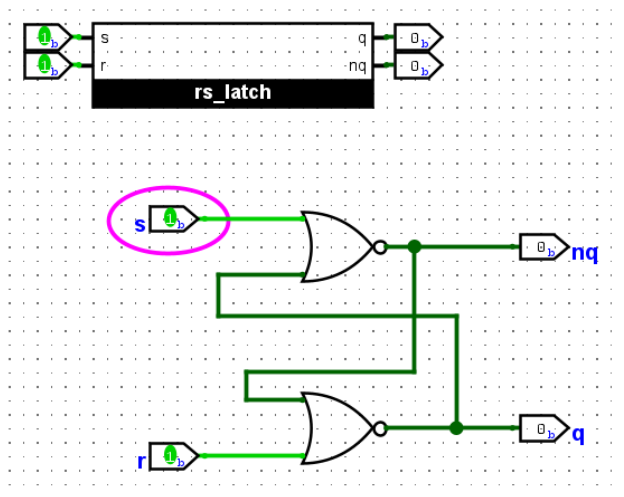
[그림 1-1] 초기 상태 (s=0, r=0): 처음에 두 입력이 모두 0이면 아직 저장된 값이 없어서 게이트 회로에는 빨간선(E), VHDL에는 U가 출력된다.



[그림 1-2] Set 동작 (s=1, r=0): s에 1을 주자 빨간색 에러가 사라지면서 q=1, nq=0으로 값이 정상적으로 들어갔다.



[그림 1-3] Reset 동작 (s=0, r=1): s=0으로 내리고 r=1을 넣었을 때 q=0, nq=1로 리셋되는 모습이다.



[그림 1-4] 금지 상태 (s=1, r=1): 두 입력에 동시에 1을 넣으면 위아래 NOR 게이트가 둘 다 0을 내보내서 q=0, nq=0이 된다.

3. NAND 게이트 기반 D Latch

3.1 회로 구성 및 동작 원리

D Latch는 앞선 RS Latch에서 1, 1 금지 입력이 들어가는 문제를 막기 위해 데이터 입력(D)에 NOT 게이트를 달아서 위아래로 항상 반대되는 값이 들어가도록 바꾼 회로다. 여기에 Enable(en) 핀을 추가해서 en=1일 때만 입력 D가 출력 q_1으로 그대로 전달되고, en=0일 때는 앞단 NAND 게이트 출력이 둘 다 1로 막혀서 D 값을 바꿔도 기존 출력이 그대로 유지된다.

입력 en	입력 D	출력 Q (q_1)	반전 출력	동작 상태
0	X (무관)	이전 값 유지	이전 값 유지	Hold (유지) - en=0이면 D를 바꿔도 출력이 바뀌지 않음
1	0	0	1	Reset 전달 - en=1일 때 D=0이 출력으로 바로 나감
1	1	1	0	Set 전달 - en=1일 때 D=1이 출력으로 바로 나감

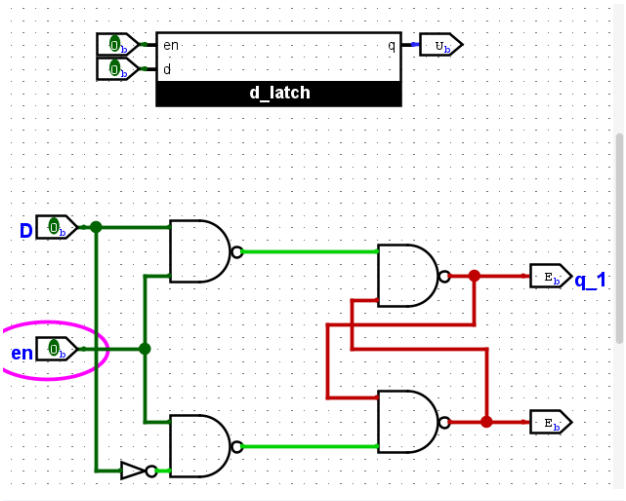
3.2 VHDL 코드 (d_latch.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

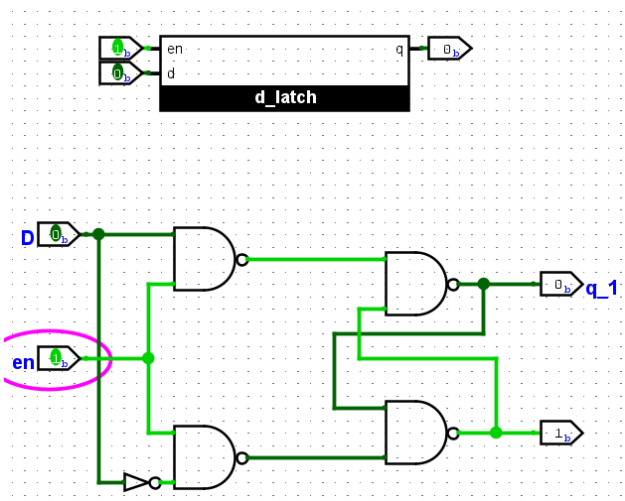
ENTITY d_latch IS
    PORT (
        en : IN  std_logic;
        d  : IN  std_logic;
        q  : OUT std_logic
    );
END d_latch;

ARCHITECTURE bhv OF d_latch IS
BEGIN
    PROCESS (en, d)
    BEGIN
        IF (en = '1') THEN
            q <= d;
        END IF;
    END PROCESS;
END bhv;
```

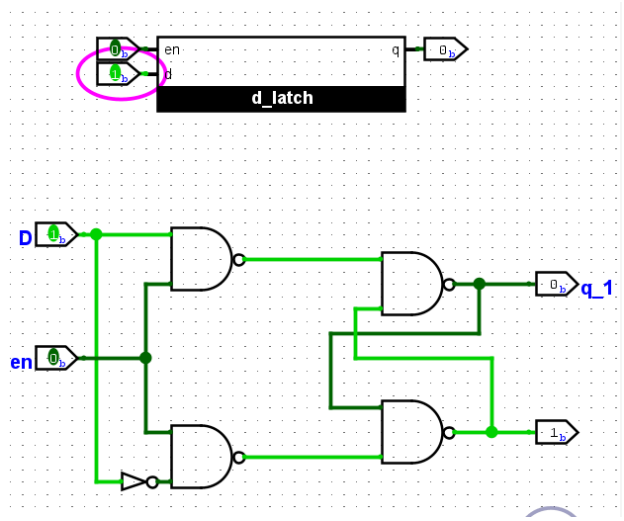
3.3 시뮬레이션 결과



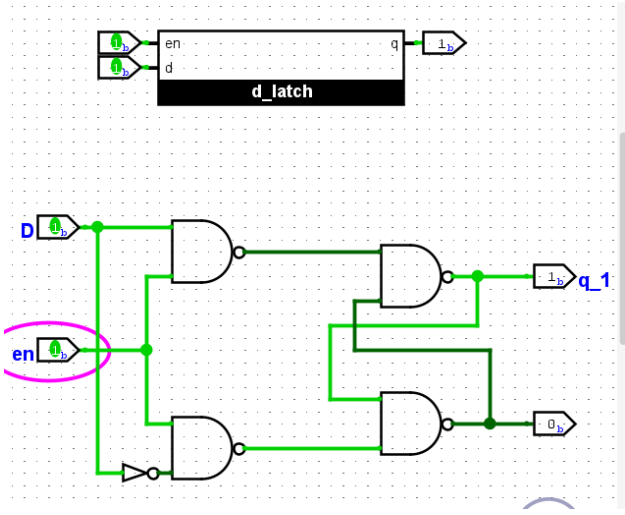
[그림 2-1] 초기 상태 (en=0, D=0): 시작할 때 en=0으로 닫혀 있으면 내부 래치 값이 정해지지 않아 E와 U가 뜬다.



[그림 2-2] en=1, D=0 입력: en을 1로 켜주자 D=0 값이 넘어가면서 에러가 풀리고 q_1=0으로 초기화되었다.



[그림 2-3] Hold 확인 (en=0, D=1): en=0으로 먼저 내린 다음 D=1로 바꿔보았는데, 출력이 바뀌지 않고 0을 그대로 유지하는 것을 확인했다.



[그림 2-4] en=1, D=1 입력: 다시 en=1을 누르자 대기하고 있던 D=1 값이 들어가서 출력 q_1=1로 바로 바뀌었다.

4. Master-Slave D Flip-Flop

4.1 회로 구성 및 동작 원리

D Latch는 en=1인 동안 입력이 바뀌면 출력도 계속 따라 바뀌기 때문에 타이밍 맞추기가 어렵다. 그래서 D Latch 2개를 앞뒤(Master-Slave)로 이어 붙이고 클럭에 NOT 게이트를 달아 서로 반대로 열리게 구성했다. 아래쪽 클럭 배선에 NOT 게이트 2개를 직렬로 두어, `clk=0`일 때는 앞쪽 Master Latch만 열려서 d_1 값을 받아두고, `clk`가 0에서 1로 올라가는 상승 에지(Rising edge) 순간에 Master는 닫히고 뒤쪽 Slave Latch가 열리면서 최종 출력이 나가도록 만들었다.

클럭 (clk)	입력 (d_1)	출력 Q	반전 출력	동작 상태
0	0 또는 1	이전 값 유지	이전 값 유지	Master Latch에만 d_1 값이 들어가고 최종 출력은 아직 안 바뀜
↑ (0 → 1)	0	0	1	클럭이 0에서 1로 올라갈 때 0이 최종 출력으로 나감
↑ (0 → 1)	1	1	0	클럭이 0에서 1로 올라갈 때 1이 최종 출력으로 나감

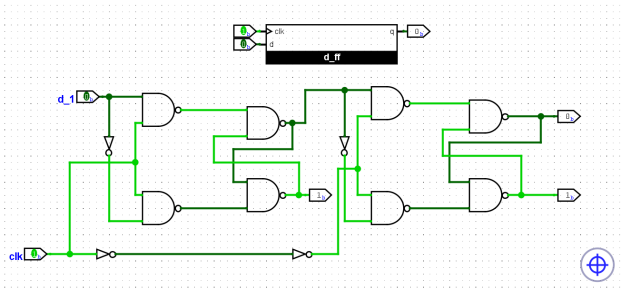
4.2 VHDL 코드 (d_ff.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

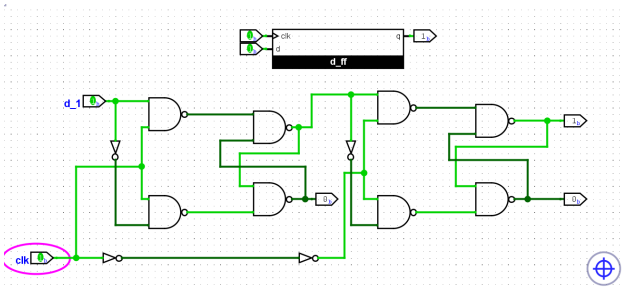
ENTITY d_ff IS
    PORT (
        clk : IN  std_logic;
        d   : IN  std_logic;
        q   : OUT std_logic
    );
END d_ff;

ARCHITECTURE bhv OF d_ff IS
BEGIN
    PROCESS (clk)
    BEGIN
        IF rising_edge(clk) THEN
            q <= d;
        END IF;
    END PROCESS;
END bhv;
```

4.3 시뮬레이션 결과



[그림 3-1] d_1=0 상태에서 클럭 상승 (clk=1): 클럭을 0에서 1로 올렸을 때 VHDL 출력 q와 게이트 회로 출력이 둘 다 0으로 맞춰졌다.



[그림 3-2] d_1=1 상태에서 클럭 상승 (clk=1): clk=0일 때 d_1=1을 미리 넣어두고 클럭을 1로 올리자 최종 출력이 1로 업데이트되었다.

5. NAND 게이트 기반 JK Flip-Flop

5.1 회로 구성 및 동작 원리

JK Flip-Flop은 J=1, K=1이 들어왔을 때 기존 출력값을 반대로 뒤집는 토글(Toggle) 기능이 들어간 회로다. 앞단에 3입력 NAND 게이트 2개를 배치해서 맨 끝 출력단(q_3, complete_q_3)의 선을 앞쪽으로 다시 끌어와 피드백시켰다. 처음 회로를 그렸을 때 피드백 선 때문에 초기값이 안 잡히고 계속 빨간 선(E)이 떠서, 오른쪽 아래 NAND 게이트를 3입력으로 바꾸고 보조 입력 핀(0)을 하나 달아서 초기 상태를 잡아주었다.

입력 J	입력 K	클럭 (CLK_1)	출력 Q (q_3)	동작 상태
0	0	0 / ↑	이전 값 유지	Hold (유지) - 입력 변화 없이 기존 출력값을 그대로 유지함
1	0	1 (↑)	1	Set (설정) - 출력이 1로 세트됨
0	1	1 (↑)	0	Reset (초기화) - 출력이 0으로 리셋됨
1	1	1 (↑)	NOT Q (반전)	Toggle (반전) - 이전 상태가 0이었으므로 반대 값인 1로 뒤집힘

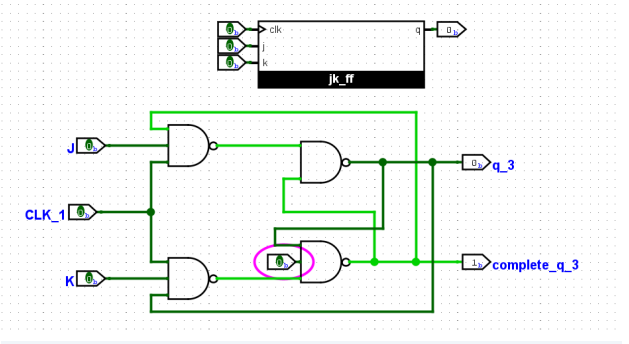
5.2 VHDL 코드 (jk_ff.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

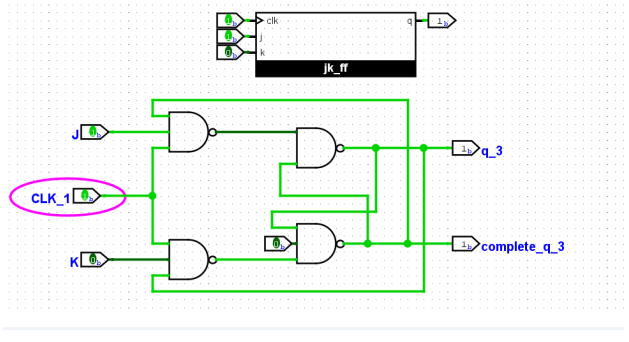
ENTITY jk_ff IS
    PORT (
        clk : IN  std_logic;
        j   : IN  std_logic;
        k   : IN  std_logic;
        q   : OUT std_logic
    );
END jk_ff;

ARCHITECTURE bhv OF jk_ff IS
    SIGNAL q_int : std_logic := '0';
BEGIN
    PROCESS (clk)
    BEGIN
        IF rising_edge(clk) THEN
            IF (j = '0' AND k = '1') THEN
                q_int <= '0';
            ELSIF (j = '1' AND k = '0') THEN
                q_int <= '1';
            ELSIF (j = '1' AND k = '1') THEN
                q_int <= NOT q_int;
            END IF;
        END IF;
    END PROCESS;
    q <= q_int;
END bhv;
```

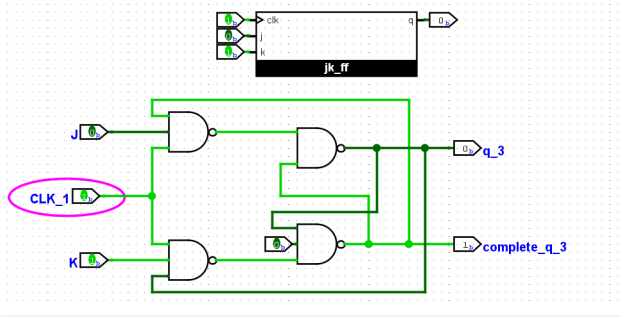
5.3 시뮬레이션 결과



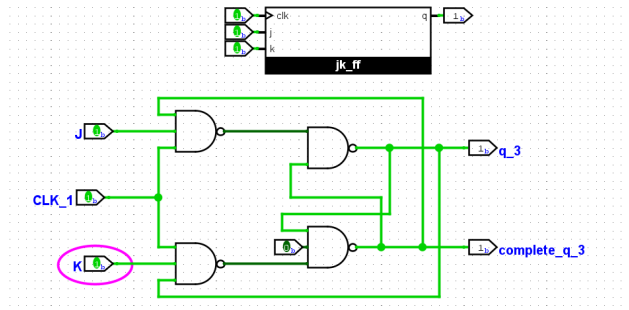
[그림 4-1] 초기 상태 및 Hold (J=0, K=0): 아래쪽 보조핀으로 초기 에러를 없앤 상태이며, J=0, K=0일 때 q_3=0이 그대로 유지된다.



[그림 4-2] Set 동작 (J=1, K=0, CLK_1=1): J=1, K=0을 넣고 클럭을 올리자 VHDL 출력 q와 게이트 출력 q_3 모두 1이 나왔다.



[그림 4-3] Reset 동작 (J=0, K=1, CLK_1=1): J=0, K=1로 바꾸고 클럭을 주었을 때 다시 q=0, q_3=0으로 리셋되는 것을 확인했다.



[그림 4-4] Toggle 동작 (J=1, K=1, CLK_1=1): 0이 저장된 상태에서 J=1, K=1을 넣고 클럭을 올리자 값이 반전되어 1이 출력되었다.

6. 비동기 Set / Clear가 포함된 D Flip-Flop

6.1 회로 구성 및 Clear 우선순위 설계

마지막 5번 회로는 3번에서 만든 Master-Slave D Flip-Flop 구조에 클럭과 상관없이 동작하는 비동기 입력 `clr`와 `set`을 추가한 것이다. 과제 조건에서 `clr=1`과 `set=1`이 동시에 들어오면 **Clear가 Set보다 우선순위**를 가져야 한다고 되어 있어서, 왼쪽 아래에 NOT 게이트 1개와 NAND 게이트 1개로 우선순위 제어 회로를 따로 만들었다.

- **우선순위 동작 원리:** `clr` 핀에 NOT 게이트를 연결한 선을 Master와 Slave의 아래쪽(n_complete_q 방향) 3입력 NAND 게이트에 연결했다. 그리고 이 NOT `clr` 선과 `set` 핀을 2입력 NAND 게이트로 묶어서 위쪽(q_4 방향) 3입력 NAND 게이트에 넣었다. 이렇게 연결하면 `clr=1, set=1`을 동시에 눌렀을 때 NOT `clr`가 0이 되면서 아래쪽 NAND는 바로 Clear 동작을 수행하고, 동시에 `set` 쪽 NAND 출력은 무조건 1로 막혀서 Clear가 이기게 된다.

입력 clr	입력 set	클럭 (clk_2)	데이터 (D_3)	출력 Q (v_q / q_4)	반전 출력	동작 설명
1	0	X (무관)	X (무관)	0	1	비동기 Clear - 클럭이 0이어도 누르는 즉시 출력이 0으로 초기화됨
0	1	X (무관)	X (무관)	1	0	비동기 Set - 클럭이 0이어도 누르는 즉시 출력이 1로 세트됨
1	1	X (무관)	X (무관)	0	1	Clear 우선순위 - 둘 다 1일 때는 clr가 우선 적용되어 출력이 0이 됨
0	0	↑ (0 → 1)	1	1	0	동기식 D-FF - 비동기 핀이 둘 다 0일 때는 클럭 상승 에지에 맞춰 D_3 값이 들어감

6.2 VHDL 코드 (d_ff_async.vhd)

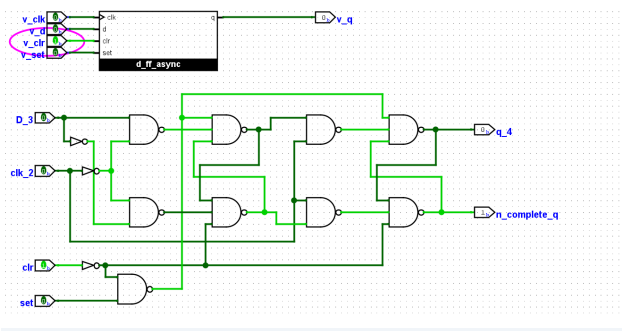
```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY d_ff_async IS
    PORT (
        clk : IN  std_logic;
        d   : IN  std_logic;
        clr : IN  std_logic;
        set : IN  std_logic;
        q   : OUT std_logic
    );
END d_ff_async;

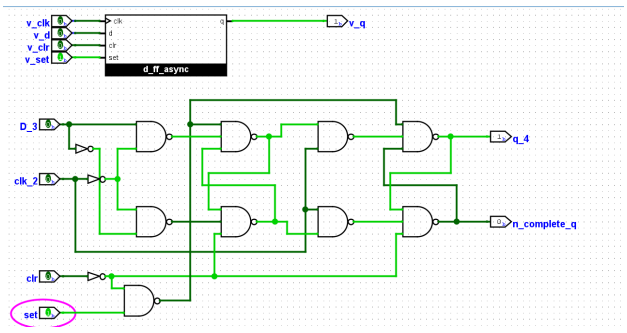
ARCHITECTURE bhv OF d_ff_async IS
    SIGNAL q_int : std_logic := '0';
BEGIN
    q <= q_int;

    PROCESS (clk, clr, set)
    BEGIN
        IF (clr = '1') THEN
            q_int <= '0';
        ELSIF (set = '1') THEN
            q_int <= '1';
        ELSIF rising_edge(clk) THEN
            q_int <= d;
        END IF;
    END PROCESS;
END bhv;
```

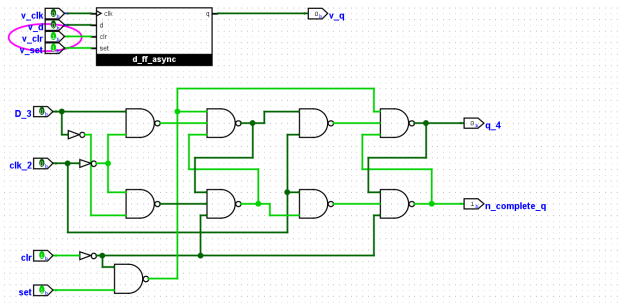
6.3 시뮬레이션 결과



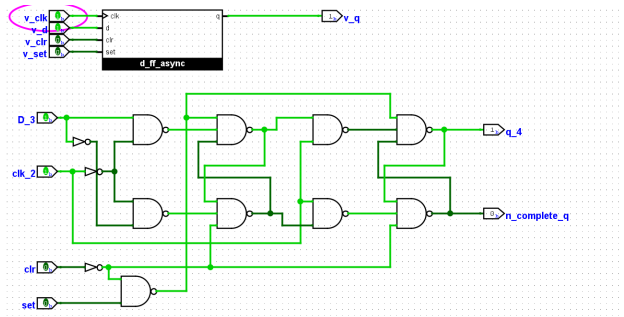
[그림 5-1] 비동기 Clear (clr=1, set=0): 클럭(clk_2)이 0인 상태에서 clr=1을 누르자 클럭과 상관없이 즉시 v_q=0, q_4=0으로 리셋되었다.



[그림 5-2] 비동기 Set (clr=0, set=1): clr=0으로 내리고 set=1을 켜자 마찬가지로 클럭 없이 바로 v_q=1, q_4=1로 바뀌었다.



[그림 5-3] 동시 입력 시 우선순위 확인 (clr=1, set=1): clr와 set을 동시에 1로 켰을 때, 우선순위 회로에 의해 set이 무시되고 v_q=0, q_4=0이 나오는 것을 확인했다.



[그림 5-4] 일반 D-FF 동작 확인 (clr=0, set=0): 비동기 핀을 모두 0으로 끈 뒤 D_3=1을 넣고 클럭을 1로 올리자 정상적으로 1이 출력되었다.

7. 고찰 및 느낀점

이번 과제를 하면서 가장 애를 먹었던 부분은 게이트로 직접 피드백 선을 연결했을 때 시뮬레이션 시작하자마자 선이 빨갛게(E) 뜨는 문제였다. 처음에는 배선을 잘못한 줄 알고 회로를 여러 번 다시 그렸는데, 래치나 플립플롭 구조상 전원이 처음 켜질 때 초기 상태가 정해지지 않아서 생기는 현상이라는 것을 알게 되었다. 특히 5번 비동기 D Flip-Flop을 만들 때는 Master 래치와 Slave 래치 양쪽의 3입력 NAND 게이트 4개에 비동기 제어선을 모두 연결해주어야만 빨간 선이 사라지고 정상적으로 초기값이 잡혔다. 또한 VHDL 코드에서는 IF (clr='1') ... ELSIF (set='1')처럼 조건문 순서만 정해주면 우선순위가 쉽게 구현되지만, 이를 실제 게이트 레벨로 똑같이 구현하려면 NOT 게이트와 NAND 게이트를 조합해서 상대 신호를 막아주는 회로가 추가로 들어가야 한다는 점을 직접 배선해 보며 확실히 이해할 수 있었다.